Exercise Sheet 08

A Thousand Arrows at Once

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (your venv must be active).
- One file per exercise, in `my_work/`. Run from the lesson folder: `python my_work/ex1_first_array.py`, so that `data/quests.csv` is found.
- Exercises 2–5 and 7 start with the **loader** from Exercise 1. Copy it from your own file and extend it: collect `paid`, `days`, `difficulty`, `monster` or `outcome` into extra lists as needed (`float(...)` for crowns and days, `int(...)` for difficulty, plain text for the rest).
- Compare with the **expected output** character by character. Rounding matters: use `.round(2)` on arrays, `round(x, 2)` on single numbers.
- Hand in: commit and push to `lesson-08`, then open a pull request. Tell Lars which loop you were glad not to write.

Exercise 1 Your first column

10 min · *ex1_first_array.py*

> **Raven from Pell, the Quartermistress:** *"The reward column of the ledger. All 400 quests. Lowest, highest, average, median, total. The Grimoire does it in a loop that takes forty seconds and sometimes skips the first row."*

GIVEN (THE LOADER)

```
import csv
import numpy as np

rewards = []
with open("data/quests.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        rewards.append(float(row["reward"]))

rewards = np.array(rewards)
```

(a) Print `type(rewards)`, `rewards.shape`, `rewards.dtype` and the first five values (`rewards[:5]`).

(b) Print the lowest, highest, average (rounded to 2 dp), median and total. Everything you need is a method on the array, except the median, which is `np.median(...)`.

EXPECTED OUTPUT

```
<class 'numpy.ndarray'>
(400,)
float64
[260. 550. 620. 670. 410.]
Lowest: 90.0
Highest: 4540.0
Average: 587.72
Median: 470.0
Total on the board: 235090.0
```

Hint: An array is a column. `(400,)` means 400 rows and no second dimension. `float64` is Excel's "Number".

Exercise 2 Dragging a formula down the column

10 min · `ex2_column_maths.py`

> **Raven from Pell, the Quartermistress:** *"In Excel I write `=L2/G2` for crowns per quest-day and drag it down 400 rows. What's the Python for 'drag down'?"*

Extend the loader so it also collects `paid`, `days` and `party_size` into arrays called `paid`, `days` and `party`. Then, **without writing a single loop**, build these arrays:

(a) `per_day` = paid / days; `per_hero` = paid / party; `per_hero_day` = paid / party / days; `with_bonus` = reward + 10 %.

(b) Print the first three of each, rounded to 2 dp.

(c) Print what the Queen's 10 % Midsummer bonus would cost in total, and the best and worst crowns per quest-day (2 dp).

EXPECTED OUTPUT

```
Crowns per quest-day: [ 65.    78.57 124.  ]
Crowns per hero: [260.  137.5 310. ]
Per hero per day: [65.   19.64 62.  ]
Rewards + 10%: [286. 605. 682.]
The Midsummer bonus would cost: 23509.0
Best crowns per quest-day: 221.67
Worst crowns per quest-day: 0.0
```

Hint: Array divided by array divides row by row: `paid / days`. Array times a number multiplies every row. That's the whole trick. The worst is 0.0 because a failed quest pays nothing.

Exercise 3 AutoFilter, but faster

10 min · `ex3_masks.py`

> **Raven from Captain Siv:** *"How many quests pay over 1,000 crowns? How many did we actually win? How many goblin jobs? I have AutoFilter open in four windows and I've lost track. Yes or no answers preferred. Numbers acceptable."*

Collect `difficulty`, `monster` and `outcome` too (`monsters = np.array(monsters)` works for text as well).

(a) `big = rewards > 1000`. Print `big[:5]`, how many are `True` (`.sum()`) and the average reward of those quests (`rewards[big]`).

(b) `success = outcomes == "Success"`. Print how many succeeded and how many did not (`~success`).

(c) `goblins = monsters == "Goblins"`. Print how many, and their average reward.

(d) Combine masks: difficulty 4 or more *and* succeeded (`&`), Trolls *or* Wyrm (`|`), over 1,000 cr *and* succeeded.

EXPECTED OUTPUT

```
[False False False False False]
Rewards over 1,000 cr: 43
Their average: 1593.02
Succeeded: 259
Did not succeed: 141
Goblin quests: 81
Average goblin reward: 312.59
Hard (4 or 5) and succeeded: 67
Trolls or Wyrm: 38
Over 1,000 cr and succeeded: 19
```

Hint: With arrays it's `&`, `|` and `~`, not `and`, `or`, `not`, and every comparison needs its own brackets: `(a > 1) & (b == 2)`.

Exercise 4 Too dangerous?

10 min · `ex4_too_dangerous.py`

> **Decree from Queen Aldara:** *"Captain Siv says the hard quests fail. Captain Siv says a great many things. Show me, one line per difficulty, in numbers a queen can read out loud."*

Collect `paid`, `days`, `difficulty` and `outcome`. Build `success` and `per_day` = paid / days.

(a) Loop over the five difficulties (a loop over 5 *levels* is fine; it's the 400 rows we don't loop over). For each: `level = difficulty == d`, then print the number of quests, the success rate (`success[level].mean() * 100`, 0 dp), and the mean and median crowns per quest-day. Use the widths in the header line.

(b) Print how many quests failed or fled, and the mean crowns per quest-day when a quest works and when it doesn't (2 dp).

EXPECTED OUTPUT

```
Difficulty  Quests  Success  Mean cr/day  Median cr/day
         1      34      88%        38.23          40.00
         2     110      75%        40.91          38.66
         3     128      62%        44.44          40.00
         4      88      57%        61.81          61.61
         5      40      42%        56.81          52.14
Failed or fled: 141 of 400
Crowns per day when it works: 67.58
Crowns per day when it doesn't: 12.03
```

Hint: A row of the table: `f"{d:>10} {level.sum():>6} {share:>6.0f}% {mean_day:>11.2f} {median_day:>13.2f}"`. So yes, Siv is right: success falls from 88 % to 42 %. But look at the crowns per day.

Exercise 5 Lean quests and rich quests

10 min · `ex5_quartiles.py`

> **Raven from Pell, the Quartermistress:** *"The Royal Auditors speak of 'quartiles'. Apparently we must say what the bottom quarter of our quests earns per day, what the top quarter earns, and which kind of quest lives where. Must we?"*

Collect `paid`, `days`, `difficulty` and `monster`; build `per_day`.

(a) `q1, q2, q3 = np.percentile(per_day, [25, 50, 75])`. Print the three cut-offs to 2 dp.

(b) `lean` = per_day at or below q1; `rich` = above q3. Print how many quests are in each.

(c) `hard` = difficulty 4 or more. Print the share of hard quests among the lean ones, among the rich ones and overall, as percentages with 1 dp. Then how many of the Wyrm quests made the top quarter.

EXPECTED OUTPUT

```
Quartile cut-offs: 16.25 cr, 42.68 cr, 72.98 cr a day
Lean quests (bottom quarter): 101
Rich quests (top quarter): 100
Hard quests among the lean ones: 30.7%
Hard quests among the rich ones: 51.0%
Hard quests overall: 32.0%
Wyrm quests in the top quarter: 2 of 7
```

Hint: `hard.mean()` on a True/False array is the share of True. For a share within a group: `(lean & hard).sum() / lean.sum()`. Biggest rewards on the board, yet only 2 of 7 Wyrm quests pay well per day.

Exercise 6 — A whole range at once

10 min · `ex6_two_dimensions.py`

> **Raven from Orrin of the Clockworks:** *"The Quartermistress's loot sheet has parties down the side and months across the top. She SUMs it both ways. Can an array do that?"*

GIVEN

```
import numpy as np

# Rows: Wren's party, Bram's party, Tamsin's party.  Columns: Jan..Jun loot in crowns.
loot = np.array([
    [620, 0, 1380, 550, 2710, 740],
    [260, 310, 0, 430, 260, 0],
    [480, 550, 670, 0, 910, 620],
])
```

(a) Print the shape, then the total per party (`axis=1`), per month (`axis=0`), and the grand total.

(b) Print Bram's April (`loot[row, column]`), Tamsin's whole row, and March for everyone (`loot[:, 2]`).

(c) Print each party's best month, how many empty-handed months each party had (`(loot == 0)` summed along the row), and a True/False array saying who did better in May than in April.

EXPECTED OUTPUT

```
(3, 6)
Per party (sum along the row): [6000 1260 3230]
Per month (sum down the column): [1360  860 2050  980 3880 1360]
Grand total: 10490
Bram in April: 430
Tamsin, all months: [480 550 670   0 910 620]
March, everyone: [1380    0  670]
Best month per party: [2710  430  910]
Empty-handed months per party: [1 2 1]
Better in May than April: [ True False  True]
```

Hint: `axis=0` squashes the rows together (you get one number per column); `axis=1` squashes the columns (one per row). Everyone gets this backwards at least once.

Exercise 7 What if? STRETCH

15 min · `ex7_what_if.py`

> **Decree from Queen Aldara:** *"Two ideas for the Privy Council. A: a party that flees gets half the reward, not a quarter. B: nobody earns under 25 crowns a quest-day, win or lose; they still walked there. What does each do to crowns per quest-day, and what does it cost the guild?"*

Collect `reward`, `paid`, `days` and `outcome`. Write a function `crowns_per_day(paid, days)` that returns all the crowns paid divided by all the days, and use it three times.

(a) Print today's crowns per quest-day.

(b) Idea A: `np.where(condition, value_if_true, value_if_false)` is `IF()` dragged down a column. Build the new paid array (fled quests get reward × 0.5, the rest keep what they were paid), print its crowns per quest-day and the extra cost.

(c) Idea B: `np.maximum(paid, 25 * days)`. Print its crowns per quest-day, its cost, and how many quests it changes.

EXPECTED OUTPUT

```
Crowns per quest-day today: 47.01
Idea A, crowns per quest-day: 51.45
Idea A costs the guild: 13,800.00 cr
Idea B, crowns per quest-day: 53.05
Idea B costs the guild: 18,795.00 cr
Quests that idea B changes: 146
```

Bug hunt Pages from the Grimoire, page 8

15 min · `bughunt/danger_bug.py`

Dalvric discovered NumPy in his last month at the guild and wrote a "danger report" for the Privy Council. It crashed, and he sent the Queen the error message by raven, by mistake. **Three** bugs: one crashes with an error message you haven't seen before (read all of it), one gives a wrong number without complaining (compare with your Exercise 5), one gets a sign wrong.

EXPECTED OUTPUT ONCE FIXED

```
Top quarter starts at: 72.98 cr a day
Hard quests that succeeded: 67
Success falls by 46 points from difficulty 1 to 5
```

One commit per bug, each message explaining what was wrong and why.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.